المدرسة العليا للمواصلات بتونس
Ecole Supérieure des Communications de Tunis
Higher School of Communications of Tunis

Module Cloud of Things

Cahier des charges

Smart Parking

Parking intelligent par vision par ordinateur

Auteurs

Eya Kammoun
Amal Trigui
Maissa Ben Mrad

Encadrant

Dr. Ing. Mohamed-Bécha Kaâniche

24 octobre 2026

Table des matières

1 Concept **2**
1.1 Contexte du projet . . . 2
1.2 Problématique . . . 2
1.3 Objectifs . . . 2

2 Clients, besoins et objectifs **2**

3 Besoins fonctionnels **3**
3.1 Caméra et réseau IoT . . . 3
3.2 Guidage vers la place libre . . . 3
3.3 Application PWA . . . 3

4 Équipement **4**

5 Choix technologiques **5**

6 Architecture **5**

7 Diagramme de déploiement **6**

8 Planning et tâches **6**
8.1 Diagramme de Gantt . . . 6
8.2 Charges . . . 6

9 Méthodologie **7**
9.1 Principes . . . 7
9.2 Processus . . . 7

10 Étude commerciale **8**
10.1 Business Model Canvas . . . 8
10.2 Politique marketing . . . 9
10.2.1 Produit . . . 9
10.2.2 Prix . . . 9
10.2.3 Promotion . . . 9
10.2.4 Place . . . 9

11 Livrables **9**

12 Limites, hypothèses et risques **10**
12.1 Limites . . . 10
12.2 Hypothèses . . . 10
12.3 Risques . . . 11

1 Concept

1.1 Contexte du projet

Dans les grandes villes, la recherche d'une place de stationnement provoque de la congestion, de la pollution et une perte de temps pour les conducteurs. Les solutions IoT classiques reposent sur un capteur infrarouge ou ultrason par place : le câblage, le coût et la maintenance augmentent avec la taille du parking.

Notre projet propose de remplacer ces capteurs par une **caméra** associée à un modèle de vision par ordinateur. Une seule caméra couvre plusieurs places, et le système ne se limite plus à savoir si une place est libre : il détecte aussi les véhicules mal placés ou en infraction, et guide le conducteur vers la place libre la plus proche. Le conducteur peut en outre réserver et payer sa place à distance.

1.2 Problématique

- Comment connaître en temps réel l'état de chaque place d'un parking, et permettre sa réservation à distance, sans installer un capteur par place ?
- Comment guider un conducteur vers une place libre sans envoyer plusieurs conducteurs vers la même place ?
- Comment détecter automatiquement les stationnements gênants (voiture à cheval sur deux places, véhicule ventouse, place spéciale occupée sans autorisation, zone interdite) tout en respectant la vie privée des usagers ?

1.3 Objectifs

- Mesurer l'occupation des places par caméra et vision par ordinateur, en remplacement des capteurs infrarouges.
- Guider le conducteur vers la place libre la plus proche, avec un plan du parking dans l'application.
- Alerter l'exploitant en cas d'anomalie (voiture à cheval sur deux places, véhicule ventouse, place spéciale, zone interdite), notamment par notification web push.
- Permettre la réservation et le paiement à distance d'une place.
- Garantir la confidentialité dès la conception (*privacy by design*) : traitement local des images, aucune plaque ni visage, aucun enregistrement vidéo.

2 Clients, besoins et objectifs

Qui adopte la solution ? Les exploitants de parkings (communes, centres commerciaux, entreprises, campus, hôpitaux) en sont les clients. Les utilisateurs sont les conducteurs et les gestionnaires du parking, qui accèdent à une même application web progressive (PWA) à travers deux interfaces distinctes selon leur rôle.

À quels besoins répond-elle ?

- Pour le conducteur : gagner du temps, connaître les places libres avant d'arriver, réserver et être guidé jusqu'à sa place.
- Pour l'exploitant : réduire le coût d'installation (une caméra à la place d'un capteur par place), mieux exploiter les places, repérer rapidement les stationnements gênants et disposer de statistiques d'occupation.

À quelles occasions ? Au quotidien, à l'arrivée au parking ou lors d'une réservation faite à l'avance, et en continu pour la supervision par l'exploitant. Le besoin est plus fort aux heures de pointe, lors d'événements et dans les zones où les places sont rares (centre-ville, hôpital, campus).

Quelles sont les limites fonctionnelles (hors périmètre) ?

- Pas de reconnaissance de plaques d'immatriculation ni d'identification de personnes.
- Pas de verbalisation automatique : le système signale les infractions à l'exploitant, qui décide de la suite.
- Pas de guidage routier dans la ville : le guidage se limite à l'intérieur du parking.
- Pas de transactions bancaires réelles : le paiement est simulé pour la démonstration.

3 Besoins fonctionnels

3.1 Caméra et réseau IoT

Une caméra installée en hauteur observe les places. Un Raspberry Pi 4 exécute localement un modèle de détection de véhicules, puis publie sur MQTT l'état des places et les événements détectés. Le système assure les fonctions suivantes :

- **Occupation** : chaque place est un polygone sur l'image ; elle est occupée si la boîte d'un véhicule la recouvre suffisamment.
- **Voiture à cheval sur deux places** : alerte lorsqu'un véhicule chevauche deux places ou dépasse du marquage.
- **Véhicule ventouse** : un chronomètre mesure la durée d'occupation de chaque place ; un événement est émis au-delà de la durée autorisée ou si le véhicule reste plusieurs jours.
- **Alerte place spéciale** : une place PMR, de recharge électrique ou réservée occupée sans réservation ou autorisation correspondante déclenche une alerte. La caméra ne voit pas les badges, la règle repose donc sur les autorisations enregistrées dans le système.
- **Infraction** : un véhicule détecté dans une zone interdite (allée, sortie) déclenche un événement.
- **Alerte** : les événements sont transmis à l'exploitant par notification web push. Un affichage ou des LED d'état par place peuvent compléter le dispositif (optionnel).

3.2 Guidage vers la place libre

Le parking est modélisé par un graphe dont les nœuds sont l'entrée, les intersections et les places, et dont les arêtes sont les allées pondérées par leur longueur. À chaque changement d'occupation, le serveur calcule le plus court chemin (Dijkstra ou A*) entre l'entrée et la place libre la plus proche. La place proposée est verrouillée temporairement (2 à 3 minutes) pour éviter d'envoyer deux conducteurs vers la même place. Si elle est prise entre-temps, le trajet est recalculé et l'application en est notifiée.

3.3 Application PWA

Une application web progressive (PWA) unique propose deux interfaces, affichées selon le rôle de l'utilisateur (conducteur ou exploitant) porté par son jeton JWT. Le serveur vérifie ce rôle à chaque appel de l'API, de sorte qu'un conducteur ne peut jamais accéder aux données d'exploitation.

Interface conducteur (front-office)

- Inscription et authentification sécurisée.

- Visualisation en temps réel des places libres sur le plan du parking.
- Guidage vers la place libre la plus proche, avec le trajet affiché sur le plan.
- Réservation d'une place, paiement à distance et historique des réservations.
- Notifications web push : réservation confirmée, fin de la durée autorisée, place à libérer.

Interface exploitant (back-office)

- Tableau de bord temps réel de l'occupation du parking.
- Liste des alertes et événements (voiture à cheval, véhicule ventouse, place spéciale, infraction), avec notifications web push.
- Configuration du parking : plan, zones des places, type de chaque place, durée maximale autorisée.
- Analytiques et indicateurs : taux d'occupation, heures de pointe, durée moyenne de stationnement, nombre d'infractions.
- Gestion des utilisateurs, des réservations et des autorisations.

4 Équipement

Le prototype repose sur le matériel suivant :

- Raspberry Pi 4 (4 Go de mémoire minimum), carte microSD de 32 Go ou plus.
- Alimentation USB-C 5 V / 3 A, dissipateur ou ventilateur pour limiter la chauffe lors de l'inférence.
- Caméra (module caméra du Raspberry Pi ou webcam USB) et support en hauteur pour une vue plongeante stable.
- Maquette de parking (carton ou bois) avec marquage des places au ruban adhésif, zone interdite et allées.
- Voitures miniatures de plusieurs couleurs.
- Éclairage LED stable pour limiter les variations de lumière.
- Câbles réseau, routeur ou point d'accès Wi-Fi.
- Serveur cloud IaaS (machine virtuelle) pour le broker MQTT, le serveur d'applications et la base de données.
- Smartphone et ordinateur pour tester l'application PWA.
- Composants optionnels : LED ou petit écran pour l'affichage de l'état des places.

Variante économique : un ordinateur portable peut remplacer le Raspberry Pi pour l'inférence, et un smartphone peut servir de caméra. L'architecture reste identique.

5 Choix technologiques

Couche	Technologie	Rôle
Vision par ordinateur	Python, OpenCV, YOLO léger (YOLOv8n ou YOLO11n, NCNN ou TFLite)	Détection des véhicules, occupation des places, détection des anomalies.
IoT	MQTT, Mosquitto, Node-RED (optionnel)	Transport des états et des événements entre le Raspberry Pi et le serveur.
Middleware	Jakarta EE 11, JAX-RS, WildFly	Règles métier, calcul des trajets, API REST, authentification OAuth2/JWT.
Base de données	MongoDB	Stockage des places, du plan (graphe), des réservations, des utilisateurs et des événements.
Front-end	PWA (HTML, CSS, JavaScript)	Application unique avec une interface conducteur et une interface exploitant, installable sur mobile et ordinateur.
Notifications	Web Push (service worker, clés VAPID)	Alertes envoyées au navigateur de l'exploitant et du conducteur.
Déploiement	Cloud IaaS, GitHub	Hébergement du broker, du serveur et de la base ; gestion du code source.

Table 1 – Choix technologiques par couche

6 Architecture

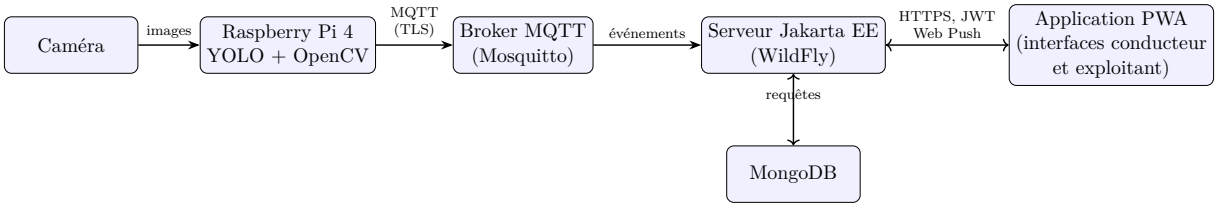


Figure 1 – Architecture globale du système

La caméra filme le parking et le Raspberry Pi analyse les images localement : seuls l'état des places et les événements détectés sont publiés sur le broker MQTT. Le serveur Jakarta EE reçoit ces messages, applique les règles métier, calcule les trajets de guidage, enregistre les données dans MongoDB et expose une API REST sécurisée par OAuth2/JWT. L'application PWA consomme cette API, affiche l'interface correspondant au rôle de l'utilisateur (conducteur ou exploitant) et reçoit les alertes par notification web push.

7 Diagramme de déploiement

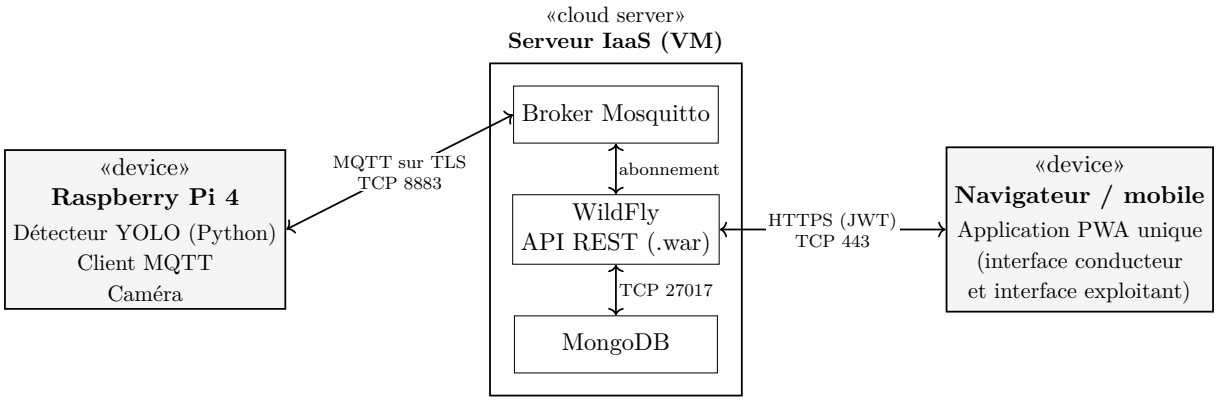


FIGURE 2 – Diagramme de déploiement UML

Le Raspberry Pi est le seul nœud installé sur site ; il communique avec le serveur cloud par MQTT sur TLS. Les navigateurs et mobiles accèdent à l'API en HTTPS avec un jeton JWT. Le broker, le serveur d'applications et la base de données sont hébergés sur une même machine virtuelle IaaS.

8 Planning et tâches

8.1 Diagramme de Gantt

Tâches	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
Planification et besoins	■									
Conception (architecture, UML)	■	■								
Maquette et installation caméra		■	■							
Modèle de détection (YOLO)			■	■	■					
Communication MQTT				■						
Base de données MongoDB					■	■				
Serveur Jakarta EE et API REST					■	■	■			
Application PWA						■	■	■		
Tests et déploiement									■	
Documentation										■

FIGURE 3 – Diagramme de Gantt

Le diagramme de Gantt présente les principales tâches du projet et leur durée, semaine par semaine. Il permet de suivre visuellement l'avancement du projet.

8.2 Charges

Le tableau suivant donne la charge de chaque tâche en hommes/jour (travail d'une personne pendant une journée).

Tâche	**Charge (hommes/jour)**
Planification et besoins	6
Conception (architecture, UML)	12
Maquette et installation caméra	9
Modèle de détection (YOLO)	27
Communication MQTT	6
Base de données MongoDB	9
Serveur Jakarta EE et API REST	27
Application PWA	30
Tests et déploiement	15
Documentation	9
Total	**150**

TABLE 2 – Charges par tâche

Pour vérifier cette estimation, nous utilisons le modèle COCOMO de base, qui calcule l'effort à partir de la taille du code. Pour un projet organique (petite équipe) d'environ 3 000 lignes de code :

$$\text{Effort} = 2{,}4 \times 3^{1{,}05} \approx 7{,}6 \text{ hommes-mois} \approx 160 \text{ hommes/jour}$$

Ce résultat est proche des 150 hommes/jour prévus, ce qui confirme que le planning est réaliste.

9 Méthodologie

Tout au long du projet, nous utiliserons l'Extreme Programming (XP), une méthode agile de développement logiciel connue pour la qualité du code produit et la satisfaction de l'équipe. Cette méthode convient bien à notre projet : il est court, l'équipe est petite et les besoins peuvent changer après chaque réunion avec l'encadrant.

9.1 Principes

Les principes de XP rejoignent ceux des méthodes agiles, mais les poussent à l'extrême. XP repose sur :

- Une grande réactivité face aux changements des besoins.
- Le travail d'équipe, représenté dans notre cas par la programmation en binôme.
- La livraison d'un travail de qualité.
- Des tests réalisés tôt et régulièrement.

9.2 Processus

Le cycle XP comprend cinq phases qui se répètent à chaque itération :

1. **Planification** : nous écrivons les *user stories* (par exemple : « en tant que conducteur, je veux être guidé vers la place libre la plus proche »), nous estimons leur durée et nous les répartissons en itérations.
2. **Conception** : nous concevons une architecture simple et claire, sans complexité inutile.
3. **Codage** : nous programmons en respectant des règles communes, en binôme sur les parties difficiles, avec une intégration continue du code sur GitHub.

4. **Tests** : chaque fonctionnalité est vérifiée par des tests unitaires, puis par des tests d'acceptation sur la maquette du parking.
5. **Écoute** : nous présentons régulièrement l'avancement à l'encadrant et nous tenons compte de ses remarques pour l'itération suivante.

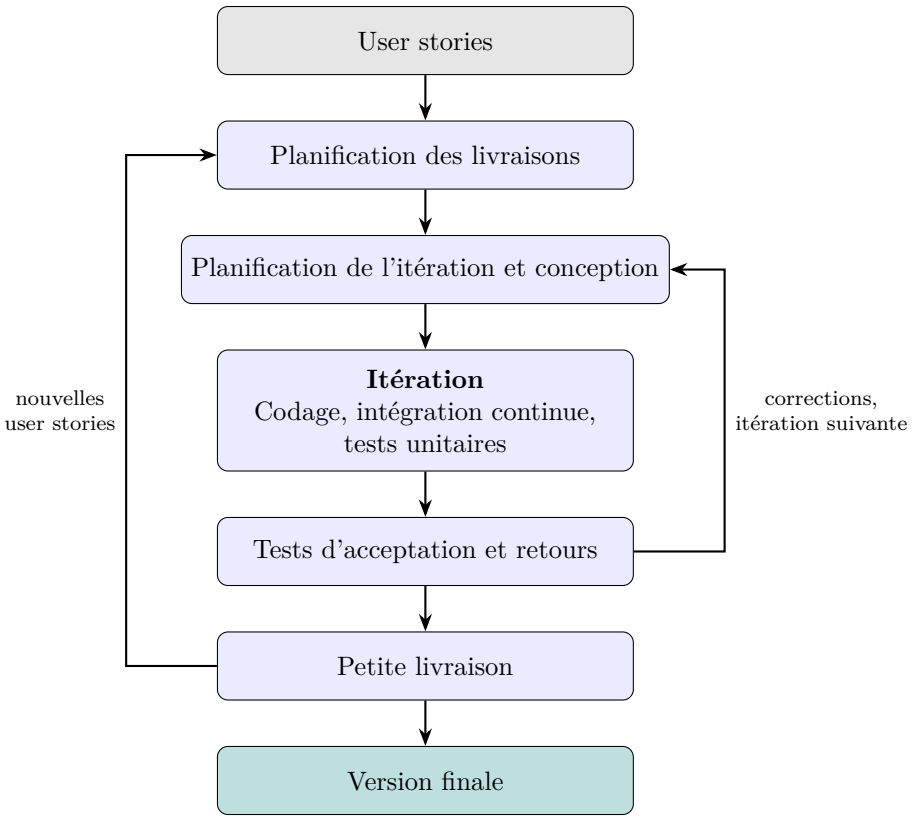


FIGURE 4 – Cycle de vie de l'Extreme Programming

10 Étude commerciale

Cette partie présente le modèle économique du projet et sa politique marketing.

10.1 Business Model Canvas

Le Business Model Canvas résume les éléments principaux du projet : proposition de valeur, clients, canaux, coûts, revenus, etc.

<table>
<tr>
<td rowspan="2">Partenaires
• Fournisseurs de caméras et de Raspberry Pi
• Hébergeur cloud
• Service de paiement en ligne
• Installateurs</td>
<td>Activités
• Développement du système
• Installation chez le client
• Maintenance</td>
<td rowspan="2">Valeur
• Une caméra au lieu d'un capteur par place
• Places libres en temps réel
• Guidage vers la place libre
• Détection des stationnements gênants
• Respect de la vie privée</td>
<td>Relations
• Assistance technique
• Formation du gestionnaire
• Mises à jour</td>
<td rowspan="2">Clients
• Gestionnaires de parkings : communes, centres commerciaux, entreprises, campus, hôpitaux
• Conducteurs</td>
</tr>
<tr>
<td>Ressources
• Modèle de détection
• Kit caméra et Raspberry Pi
• Serveur et application</td>
<td>Canaux
• Vente directe
• Salons et démonstrations
• Site web et réseaux sociaux</td>
</tr>
<tr>
<td colspan="3">Structure des coûts
Matériel, hébergement cloud, développement, installation, maintenance et marketing.</td>
<td colspan="3">Sources de revenus
Vente du kit et installation, abonnement mensuel, commission sur les réservations.</td>
</tr>
</table>

FIGURE 5 – Business Model Canvas

10.2 Politique marketing

Notre politique marketing vise à apporter de la valeur aux clients tout en assurant une bonne présence sur le marché. Elle repose sur les points suivants :

10.2.1 Produit

- Un système de parking intelligent basé sur une caméra et la vision par ordinateur.
- Une application unique pour les conducteurs et les gestionnaires de parking.
- Une installation simple : une caméra couvre plusieurs places.

10.2.2 Prix

- Un prix compétitif, adapté à la taille du parking (nombre de caméras).
- Un abonnement mensuel qui comprend la maintenance et les mises à jour.
- Une application gratuite pour les conducteurs.

10.2.3 Promotion

- Des démonstrations et des vidéos de présentation du prototype.
- Une présence sur les réseaux sociaux, notamment LinkedIn.
- Des partenariats avec des communes et des centres commerciaux.

10.2.4 Place

- Le service est accessible depuis un navigateur web ou un smartphone, sans téléchargement.
- La vente se fait directement auprès des gestionnaires de parkings ou par des installateurs partenaires.

11 Livrables

- **Cahier conceptuel** : ce document présente de manière détaillée et structurée les spécifications et les services à fournir.

- **Code source sur GitHub** : l'API, l'application PWA et la passerelle IoT.
- **Documentation technique** : installation et fonctionnement du système.
- **Manuel utilisateur** : guide d'utilisation de l'application.
- **API testable avec Postman**.
- **Supports marketing et dossier de presse**.
- **Vidéos de démonstration** : vidéos de 5 minutes maximum qui présentent la solution.

12 Limites, hypothèses et risques

12.1 Limites

- La précision dépend de la lumière et de l'angle de la caméra.
- La puissance du Raspberry Pi limite la fréquence d'analyse (environ une image toutes les 1 à 2 secondes).
- Les zones des places doivent être configurées pour chaque parking.
- Le système nécessite une connexion Internet ; sans elle, les alertes et les mises à jour sont retardées.
- La caméra ne peut pas vérifier les badges (PMR, par exemple) : les alertes de place spéciale reposent sur les autorisations enregistrées.

12.2 Hypothèses

- Le développement du front-end, du middleware et de la partie IoT est mené en parallèle ; la période de mobilisation de chaque partie suit le planning de la section dédiée.
- Le client dispose d'un parking au marquage lisible, d'une alimentation électrique pour le Raspberry Pi et d'une connexion Internet.
- Une session de formation de l'exploitant à l'interface exploitant de l'application est prévue à la livraison.

12.3 Risques

Risque	Conséquence	Parade
Variation de lumière ou mauvais angle de caméra	Détections erronées, occupation fausse	Éclairage stable, ajustement du modèle sur des images de la maquette, seuils de confiance.
Puissance insuffisante du Raspberry Pi	Fréquence d'analyse réduite, retards	Modèle YOLO léger, une image toutes les 1 à 2 secondes, exécution sur ordinateur en repli.
Zones de places mal configurées ou caméra déplacée	Occupation et alertes incorrectes	Outil de calibration des polygones, vérification à l'installation.
Perte de la connexion Internet	Alertes et mises à jour retardées	File locale des événements sur le Raspberry Pi et renvoi à la reconnexion.
Atteinte à la vie privée	Perte de confiance, non-conformité	Privacy by design : traitement local, aucun enregistrement vidéo, aucune plaque ni visage, durée de conservation limitée.

Table 3 – Risques et parades